Темы рефератов: цветовая информация и скалярные поля в CloudCompare и Python

Каждая из 10 тем посвящена одной задаче, которую студент самостоятельно решает в двух средах: через интерфейс CloudCompare и кодом Python. Обе реализации используют одинаковые исходные файлы и согласованные параметры. Затем студент сопоставляет результаты. Экспорт из одной среды служит для проверки, а основная операция в другой выполняется заново по исходным данным.

Аудитория — студенты, знакомые с облаками точек, CloudCompare и базовым Python. Регламент — 10–12 минут, включая 3–4 минуты на показ обеих реализаций и сравнение. Темы 1–4 охватывают RGB-данные, 5–9 — скалярные поля, тема 10 связывает числовое поле с сохранённой окраской.

Для всех заданий используйте небольшие синтетические наборы в локальных координатах порядка единиц или десятков метров. У каждой точки должен быть уникальный целочисленный ID, заданный в исходном файле и сохраняемый при обработке; в CloudCompare его можно импортировать как скалярное поле. RGB храните целыми числами 0–255. Для обмена достаточно ASCII с явно описанными столбцами: CloudCompare поддерживает XYZ, RGB и несколько скалярных полей в этом формате. См. поддерживаемые форматы.

Сохранённые RGB-цвета, числовые значения скалярного поля и цветовая шкала его отображения — разные сущности. В каждой работе обозначайте, какая из них изменяется.

1. **Чтение и повторное сохранение RGB-облака: порядок каналов и сохранность атрибутов**

   Учебная цель: научиться одинаково интерпретировать структуру файла и проверять результат обмена данными.

   Инструменты: CloudCompare; NumPy для чтения, перестановки столбцов и сохранения; Matplotlib для просмотра RGB.

   Основные вопросы:

   - Как назначение столбцов определяет координаты и цвет точки?
   - Как распознать перестановку R и B на контрольном наборе?
   - Как проверить данные после сохранения независимо от порядка строк?

   Общие исходные данные: 300 точек, по 100 в красной, зелёной и синей группах; столбцы `X Y Z B G R ID`, координаты в метрах. Контрольные цвета — (255, 0, 0), (0, 255, 0), (0, 0, 255).

Задание в CloudCompare: открыть ASCII, правильно назначить столбцы B, G, R; включить RGB-отображение; сохранить XYZRGB и ID в новый ASCII и открыть повторно. На копии показать эффект ошибочного назначения каналов. Возможности импорта и экспорта описаны в таблице форматов.

Аналогичное задание в Python: прочитать исходник через numpy.loadtxt, собрать массив RGB в правильном порядке, вывести облако и выполнить цикл записи и чтения через numpy.savetxt. Подготовить воспроизводимый скрипт.

Сравнение результатов: после сопоставления по ID оба выходных файла содержат 300 точек и точные контрольные RGB. Максимальное отклонение любой координаты от исходника — не более 10^{-5} м при записи шести знаков после запятой; этот запас учитывает округление записи и представления локальных координат. Ошибочная перестановка меняет местами красную и синюю группы.

Обязательная демонстрация: исходные столбцы → назначение и сохранение в CloudCompare → ключевой код и запуск Python → два повторно открытых результата → таблица проверки ID, XYZ и RGB.

2. **Линейное растяжение уровней RGB и ограничение крайних значений**

Учебная цель: воспроизвести одно преобразование контраста в интерфейсе и в массиве цветов.

Инструменты: CloudCompare; NumPy для арифметики каналов; Matplotlib для облака и гистограмм.

Основные вопросы:

- Как входной и выходной диапазоны задают преобразование цвета?
- Какие значения теряют различимость при ограничении диапазона?
- Как отделить округление RGB от ошибки алгоритма?

Общие исходные данные: 500 точек с XYZ, RGB и ID; основные значения каналов — 60–160, контрольные — 40, 60, 110, 160 и 200. Координаты — в метрах, каналы — безразмерные целые числа.

Задание в CloudCompare: на копии облака применить `Edit > Colors > Levels` ко всем каналам: вход 60–160, выход 0–255. Сохранить результат. Параметры Levels описаны в официальном руководстве, с. 54.

Аналогичное задание в Python: из исходного RGB вычислить `c′ = clip((c − 60) × 255 / 100, 0, 255)`, затем округлить и перевести в целочисленный RGB. До арифметики перейти к вещественному типу. Использовать numpy.clip; скрипт должен сохранить цвета и построить изображения и гистограммы до и после.

Сравнение результатов: по ID проверить сохранность XYZ и поканальные разности RGB. Для контрольных значений ожидаются примерно 0, 0, 128, 255, 255; допустимое

расхождение двух реализаций — одна единица канала из-за способа округления. Большее расхождение требует проверки диапазонов и выбранных каналов.

Обязательная демонстрация: исходник → параметры Levels и обработка → формула в коде и запуск → результаты с одинаковым ракурсом → максимальная разность RGB и пример потери деталей на краях.

3. **Выделение цветовой группы по индексу преобладания зелёного канала**

   Учебная цель: построить числовой признак из RGB и получить одинаковую выборку точек.

   Инструменты: CloudCompare; NumPy для индекса и булевой маски; Matplotlib для исходного облака и выборки.

   Основные вопросы:

   - Как построить индекс `I = 2G - R - B` из цветовых каналов?
   - Почему для вычислений нужен тип, допускающий отрицательные значения?
   - Почему цветовой признак сам по себе не является доказательством класса объекта?

   Общие исходные данные: XYZRGB и ID для трёх групп по 100 точек: RGB (40, 120, 40), (120, 40, 40), (80, 80, 80). Соответствующие безразмерные индексы — 160, −80 и 0. Порог отбора — `I ≥ 100`.

   Задание в CloudCompare: через `Edit > Colors > Convert to scalar field` получить R, G, B; через `Edit > Scalar fields > Arithmetic` вычислить `(G + G) - R - B`; отфильтровать поле в диапазоне 100–510. Извлечение каналов описано в руководстве, с. 56, арифметика — в разделе о скалярных полях, отбор — в Filter by value.

   Аналогичное задание в Python: вычислить тот же индекс из исходного RGB в знаковом или вещественном типе; применить булеву маску NumPy ко всем атрибутам. Скрипт должен сохранить поле и выборку, вывести их размеры и изображение.

   Сравнение результатов: значения I совпадают точно на заданных целочисленных данных; обе выборки содержат одинаковые 100 ID первой группы. RGB и координаты выбранных точек сохранены. Проверять состав ID, а не только число точек.

   Обязательная демонстрация: три группы → извлечение каналов, арифметика и фильтрация в CloudCompare → ключевой код и запуск → совпадение индексов и множеств ID.

4. **Перенос RGB-цветов на другое облако по ближайшей точке**

   Учебная цель: воспроизвести геометрическое правило назначения цветов и проверить однозначность соответствия.

   Инструменты: CloudCompare; NumPy для поиска ближайших точек; Plotly для интерактивного просмотра источника и окрашенного получателя.

Основные вопросы:

- Как различаются облако — источник цвета и облако-получатель?
- Почему ближайшая точка должна определяться в общей системе координат?
- Что происходит при нескольких равноудалённых соседях и на границе цветовых участков?

Общие исходные данные: источник — сетка 20 × 20 точек с шагом 0.25 м, XYZRGB и ID; три цветовых участка имеют известные цвета. Получатель — такая же сетка, смещённая на (0.02, 0.01, 0) м, с XYZ и собственным ID, без RGB. Для каждой точки получателя ближайший источник однозначен.

Задание в CloudCompare: выбрать оба облака и выполнить `Edit > Colors > Interpolate from another entity`; получатель должен исходно не иметь цветов. Проверить и сохранить его RGB. Описанный метод копирует цвет ближайшей точки; см. руководство, с. 56.

Аналогичное задание в Python: по двум исходным массивам XYZ вычислить попарные квадраты расстояний, выбрать ближайший источник и скопировать его RGB. Подготовить скрипт с сохранением результата и двумя наборами точек в Plotly Scatter3d, показывая ID и RGB в подсказках.

Сравнение результатов: по ID получателя проверить точное совпадение всех RGB между реализациями и с ожидаемыми цветами соответствующих узлов сетки. XYZ получателя сохраняются с допуском 10^{-5} м. Полный перебор допустим для 400 × 400 точек; способ масштабирования можно обсудить без реализации.

Обязательная демонстрация: цветной источник и бесцветный получатель → перенос в CloudCompare → код поиска и запуск → интерактивный просмотр Python → число несовпадающих RGB, ожидаемо ноль.

5. **Создание поля высоты и единая цветовая шкала для нескольких облаков**

Учебная цель: получить поле из координат и обеспечить одинаковый смысл цветов в разных представлениях.

Инструменты: CloudCompare; NumPy для поля Z; Matplotlib для общей нормировки и цветовой шкалы.

Основные вопросы:

- При каких условиях координату Z можно интерпретировать как высоту?
- Почему независимые диапазоны шкал искажают визуальное сравнение облаков?
- Как согласовать значения, опорные цвета и легенды в двух средах?

Общие исходные данные: два облака XYZ и ID в общей системе координат, по 500 точек; диапазоны Z — 0–10 и 5–15 м. В обоих есть точки с Z = 5, 7.5 и 10 м; RGB отсутствует.

Задание в CloudCompare: создать поле через Export coordinate(s) to SF(s). Сравнить независимые относительные шкалы и общую абсолютную: 0 м — синий, 7.5 м — белый, 15 м — красный; настроить её в Color Scales Manager.

Аналогичное задание в Python: создать поле из исходного столбца Z; построить оба облака сначала с отдельными диапазонами, затем с общей Normalize(0, 15) и шкалой из тех же RGB-узлов через LinearSegmentedColormap. Подготовить скрипт, сохраняющий поле и рисунки.

Сравнение результатов: по ID поле каждой реализации совпадает с Z в пределах 10^{-5} м. Для общей шкалы контрольным высотам соответствуют положения 1/3, 1/2 и 2/3; равные высоты получают одинаковый цвет в обоих облаках каждой среды. Сопоставить узлы шкалы и легенды, учитывая дискретизацию; сравнение пикселей снимков экрана не является числовой проверкой поля.

Обязательная демонстрация: исходные облака → создание поля и смена шкал в CloudCompare → код нормировки и запуск → две пары изображений → таблица контрольных высот и положений на шкале.

6. **Пороговая фильтрация скалярного поля с проверкой границ интервала**

Учебная цель: получить одинаковое подмножество точек по числовому условию.

Инструменты: CloudCompare; NumPy для масок и проверки ID; Matplotlib для просмотра результата.

Основные вопросы:

- Как выбрать поле, по которому выполняется отбор?
- Чем изменение диапазона отображения отличается от создания выборки?
- Как включение границ влияет на состав результата?

Общие исходные данные: XYZ, ID и синтетическое поле `intensity` в условных единицах: по 100 точек со значениями 10, 50 и 90, ещё три точки со значениями 40, 60 и 61. Всего 303 точки; RGB не нужен. Условие — $40 \le \texttt{intensity} \le 60$.

Задание в CloudCompare: выбрать intensity и применить Edit > Scalar fields > Filter by value с границами 40 и 60, сохранив отдельное облако с ID и исходным полем.

Аналогичное задание в Python: прочитать исходный набор, построить включающую обе границы булеву маску, применить её одновременно к XYZ, ID и intensity. Скрипт должен сохранить выборку, вывести её размер и показать пространственное распределение отобранных точек.

Сравнение результатов: обе выборки должны содержать 102 точки и одинаковые ID: всю группу со значением 50 и две граничные точки. Отдельно проверить исключение 61 и сохранность атрибутов. Число и ID сравниваются точно; для исходных целых значений поля числовая погрешность не требуется.

Обязательная демонстрация: исходное распределение → фильтр и новое облако в CloudCompare → код маски и запуск → проверка 102 ID и трёх точек у границы.

7. **Вертикальное отклонение от заданной наклонной плоскости как вычисляемое поле**

   Учебная цель: построить одно знаковое поле через последовательность арифметических операций и векторную формулу.

   Инструменты: CloudCompare; NumPy для арифметики и статистик; Plotly для отображения знака отклонений.

   Основные вопросы:

   - Как выразить вертикальное отклонение через XYZ и уравнение плоскости?
   - Чем оно отличается от кратчайшего расстояния до наклонной плоскости?
   - Как проверить знак, масштаб и единицы нового поля?

   Общие исходные данные: 300 точек XYZ и ID в метрах: по 100 на плоскости `z = 0.1x + 1`, на 0.05 м выше и на 0.05 м ниже неё; X и Y — в пределах 0–5 м, RGB отсутствует.

   Задание в CloudCompare: экспортировать X и Z в скалярные поля; через `Edit > Scalar fields > Arithmetic` последовательно получить `0.1X`, `0.1X + 1`, затем `dz = Z - (0.1X + 1)`. Если нужны отдельные постоянные поля, создать их через `Add constant SF`. Возможности описаны в разделе о скалярных полях и руководстве, с. 80. Сохранить dz и применить шкалу: −0.05 — синий, 0 — белый, +0.05 — красный.

   Аналогичное задание в Python: по исходному XYZ векторно вычислить ту же формулу, среднее и RMSE — корень из среднего квадрата dz. Скрипт должен сохранить поле и вывести его в Plotly с теми же тремя узлами шкалы и фиксированными пределами.

   Сравнение результатов: сопоставить dz по ID с обоими вычислениями и ожидаемыми значениями 0, +0.05, −0.05 м; максимальная ошибка — 10^{-5} м для данного масштаба координат и ASCII с шестью знаками. Дополнительно проверить среднее около нуля и RMSE около 0.040825 м. Смена шкалы не должна менять dz.

   Обязательная демонстрация: исходник и плоскость на схеме → цепочка арифметики в CloudCompare → формула в коде и запуск → цветовые представления → ошибки поля и контрольные статистики.

8. **Нормализация скалярного поля и влияние экстремального значения на диапазон**

   Учебная цель: выполнить одинаковое преобразование числовых значений и объяснить его чувствительность к составу данных.

   Инструменты: CloudCompare; NumPy для минимума, максимума и нормализации; Matplotlib для распределений.

   Основные вопросы:

- Как вычисляется нормализация `u = (s − min(s)) / (max(s) − min(s))` ?
- Как одно экстремальное значение меняет представление основной группы?
- Что следует делать при нулевой ширине диапазона?

Общие исходные данные: набор A из 300 точек XYZ, ID и конечных значений s в условных единицах от 20 до 80, включая контрольные 20, 50, 80. Набор B содержит те же точки и одну дополнительную со значением 1000. RGB не требуется.

Задание в CloudCompare: определить минимум и максимум s для каждого набора; через `Arithmetic` создать новое поле u вычитанием минимума и делением на ширину диапазона. При необходимости использовать постоянные поля. Сохранить исходное s и вычисленное u; изменение одной цветовой шкалы не заменяет вычисление. Арифметика полей описана в официальном введении.

Аналогичное задание в Python: независимо вычислить минимум, максимум и u по исходным A и B. Подготовить скрипт с сохранением полей и гистограммами u. При постоянном s заранее выявлять нулевой знаменатель; в обеих средах принять явное правило для этого случая — создать u = 0.

Сравнение результатов: по ID значения u должны совпасть в пределах 10^{-6}. В A контрольные результаты — 0, 0.5, 1; в B — 0, 30/980, 60/980, а дополнительная точка получает 1. Допуск учитывает округление вещественного поля и запись шести знаков. Применять к обеим гистограммам одинаковые интервалы на 0–1; отдельно проверить постоянное поле.

Обязательная демонстрация: A и B → вычисление u в CloudCompare → код и запуск → таблица контрольных значений и гистограммы → объяснение сжатия основной группы и правила для постоянного поля.

9. **Поле расстояний до ближайших точек опорного облака**

Учебная цель: сопоставить расчёт CloudCompare с независимым вычислением евклидовых расстояний.

Инструменты: CloudCompare; NumPy для попарных расстояний и поиска минимумов; Matplotlib для представления поля.

Основные вопросы:

- Как роли сравниваемого и опорного облаков задают направление расчёта?
- Почему расстояние до ближайшей точки отличается от расстояния до непрерывной поверхности?
- Как отделить предварительную оценку от окончательного результата?

Общие исходные данные: опорная сетка 20 × 20 точек с шагом 0.25 м на Z = 0; сравниваемая сетка имеет те же XY и Z = 0.10 м. В обоих исходных файлах — XYZ и ID; RGB не нужен.

Задание в CloudCompare: выбрать оба облака, назначить роли и выполнить `Tools > Distances > Cloud/Cloud dist.` без локальной модели и без ограничения максимального расстояния; нажать Compute для окончательного расчёта. Сохранить полученное поле сравниваемого облака. Порядок работы и смысл метода описаны в Cloud-to-Cloud Distance.

Аналогичное задание в Python: для каждой точки сравниваемого исходного набора вычислить минимум евклидовых расстояний до всех точек опорного. Подготовить скрипт с сохранением расстояний, визуализацией и проверкой. Для этого небольшого набора достаточно операций NumPy с broadcasting, без дополнительных библиотек.

Сравнение результатов: по ID сравниваемых точек все расстояния должны равняться 0.10 м; максимальное отклонение от контроля и между реализациями — 10^{-5} м при локальных координатах и записи шести знаков. Перестановка строк опорного массива не должна менять расстояния. Для визуализации задать в обеих средах одну линейную шкалу 0–0.20 м.

Обязательная демонстрация: два исходника → выбор ролей и полный расчёт в CloudCompare → ключевой код и запуск → одинаковые шкалы → таблица расстояний и максимальная ошибка.

10. **Преобразование скалярного поля в сохранённые RGB-псевдоцвета**

Учебная цель: независимо получить RGB из числового поля и оценить потери при кодировании цвета.

Инструменты: CloudCompare; NumPy для нормировки и назначения RGB; Matplotlib для просмотра рассчитанной окраски.

Основные вопросы:

- Чем цветовое отображение поля отличается от записи RGB в данные?
- Как диапазон и дискретность шкалы определяют получаемые цвета?
- Почему по RGB нельзя гарантированно восстановить исходное поле с прежней точностью?

Общие исходные данные: 500 точек XYZ, ID и s в условных единицах от 0 до 100; контрольные значения — 0, 25, 50, 75, 100. Исходных RGB нет.

Задание в CloudCompare: настроить линейную серую шкалу от чёрного при s = 0 до белого при s = 100, без логарифмической нормировки; установить K = 256 уровней в параметре Steps. Этот параметр отличается от опорных узлов шкалы; см. пояснение разработчика CloudCompare. Выполнить `Edit > Scalar fields > Convert to RGB` и сохранить XYZ, ID, исходное s и RGB. Преобразование учитывает параметры отображения; см. официальное руководство, раздел 2.2.23.

Аналогичное задание в Python: из исходного s самостоятельно вычислить `g = round(255 × s / 100)`, назначить R = G = B = g, сохранить RGB вместе с s и отобразить облако.

Подготовить скрипт; экспорт CloudCompare загружать только после собственного расчёта, для сравнения.

Сравнение результатов: по ID проверить сохранность s с допуском 10^{-6} и равенство трёх каналов у каждого результата. Идеальные контрольные уровни серого — 0, 64, 128, 191, 255. Допуск между реализациями обосновать шагом шкалы и округлением: для K равномерных уровней серого ориентир — `ceil(255 / (K - 1)) + 1` единиц канала, то есть 2 при K = 256. Зафиксировать фактическую дискретизацию установленной версии; для другой схемы уточнить оценку. Показать максимальную поканальную ошибку, а не требовать совпадения снимков экрана.

Обязательная демонстрация: поле и шкала → преобразование в RGB в CloudCompare → ключевой код независимого расчёта и запуск → две окраски → таблица контрольных RGB и объяснение расхождений.

Общие рекомендации по подготовке

Реферат и доклад. Последовательность: задача → общие данные → метод → реализация в CloudCompare → реализация в Python → сравнение → выводы и источники. Укажите, какие операции соответствуют друг другу, что совпало, какова максимальная ошибка и чем объясняются различия. Приведите 2–4 содержательных источника, включая официальную документацию. Отчёт о выполнении должен содержать исходные файлы, результат каждой реализации и таблицу проверки.

Презентация. Вариант на 7 слайдов и 11 минут:

Слайд	Содержание	Время
1	Название, задача и учебная цель	0:30
2	Общие данные, атрибуты, единицы и контрольный пример	1:00
3	Метод и соответствие операций двух реализаций	1:30
4	Параметры CloudCompare и ключевой фрагмент Python	1:30
5	Демонстрация обеих сред и первичная проверка	3:30
6	Таблица сравнения, ошибка и объяснение различий	2:00
7	Выводы, ограничения и источники	1:00

На демонстрацию можно отвести 1:30 для CloudCompare, 1:15 для кода и запуска Python и 0:45 для сопоставления. Файлы и окна откройте заранее; подготовку данных и установку библиотек выполните до выступления. В каждой теме обязательны показ основной операции через интерфейс, объяснение ключевого кода и его запуск. Полный скрипт или notebook приложите к работе; на слайд достаточно вынести 5–12 существенных строк.

Иллюстрации и проверка. Для сравнения согласуйте ракурс, масштаб, размер точек, диапазон, направление шкалы и её RGB-узлы; подпишите единицы, число точек и происхождение окраски. В Matplotlib при проверке RGB отключите затенение по глубине через `depthshade=False`; см. Axes3D.scatter. Итоговая таблица должна содержать ID, контрольное значение, оба результата и разность; для выборок — число и состав ID. Приведённые допуски относятся к заданным

небольшим локальным наборам. При другом масштабе данных или точности экспорта пересчитайте их заранее, исходя из округления записи и вычислений.

Подготовка среды и воспроизводимость.

- Запишите версии CloudCompare, Python и библиотек и проверьте нужные операции в установленной сборке. Ссылки на руководства CloudCompare 2.1 и 2.6.1 описывают функции; расположение пунктов, диалоги и настройки дискретизации могут отличаться в другой версии. Для переноса RGB подтвердите правило ближайшей точки; для преобразования поля в RGB зафиксируйте шкалу и число её уровней. Не подменяйте согласованные операции другими алгоритмами.
- В заданиях 3, 7 и 8 сохраняйте промежуточные поля с понятными именами. Константы для Arithmetic можно представить отдельными полями через Add constant SF; заранее проверьте режим сохранения результата и работайте с копией облака.
- Сохраните исходные данные отдельно от двух результатов. Для случайной генерации зафиксируйте seed, например 42, и передайте обеим реализациям один сохранённый набор. Обработку каждой темы начинайте с исходника; проверку результатов выполняйте отдельным шагом.
- При обмене ASCII укажите порядок столбцов, разделитель и точность: для предложенных задач достаточно шести знаков после запятой для XYZ и вещественных полей, целой записи для RGB и ID. Сопоставляйте точки по исходным ID; при фильтрации переносите одну маску на все атрибуты.
- Для Python используйте NumPy и указанную библиотеку визуализации; дополнительные вычислительные пакеты этим заданиям не нужны. Если выбран notebook, добавьте Jupyter как среду запуска и проверьте выполнение всех ячеек после перезапуска ядра. Обычный скрипт также подходит.
- Проверьте повторный запуск, контрольные числа и время демонстрации. Совпадение отдельных контрольных значений дополните проверкой всего поля или множества ID; готовый результат одной среды не должен становиться входом основной операции другой.

Резерв на случай сбоя. Подготовьте скриншоты основных операций, короткую запись обеих демонстраций, выходные файлы и таблицу сравнения. Для Plotly сохраните автономный HTML со встроенным Plotly.js; см. экспорт интерактивного HTML. Резервные материалы должны соответствовать тем же входным данным и параметрам.